Análise de Eficácia de Testes com Teste de Mutação

Relatório de atividade prática

Disciplina: Teste de Software

Aluno: João Vitor Pedersoli Rajão

Matrícula: não informada

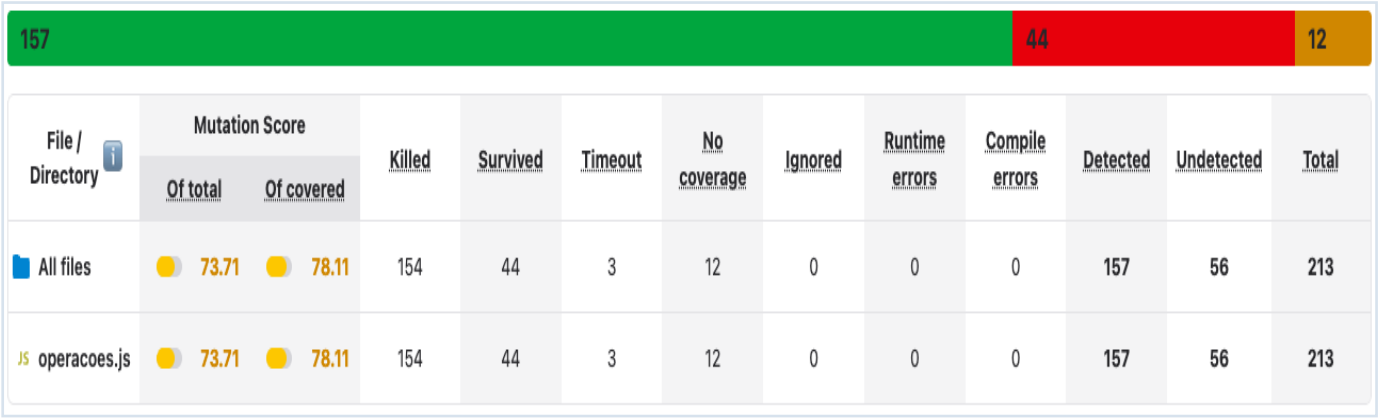
1. Análise inicial

Projeto: operacoes-mutante, biblioteca JavaScript com 50 funções e testes Jest.
A suíte inicial passou integralmente: 50 testes em 1 arquivo.

Linhas cobertas	Ramos cobertos	Mutação	Sobreviventes
98,64%	58,82%	73,71%	44

A cobertura de linhas elevada indica execução do código, mas não garante que os testes observem erros de lógica. O Stryker gerou 213 mutantes: 154 mortos, 3 por timeout, 44 sobreviventes e 12 sem cobertura. O descompasso é explicado por entradas ausentes (vazio, zero, igualdade, compostos) e asserções permissivas.

Evidência - primeira execução do StrykerJS



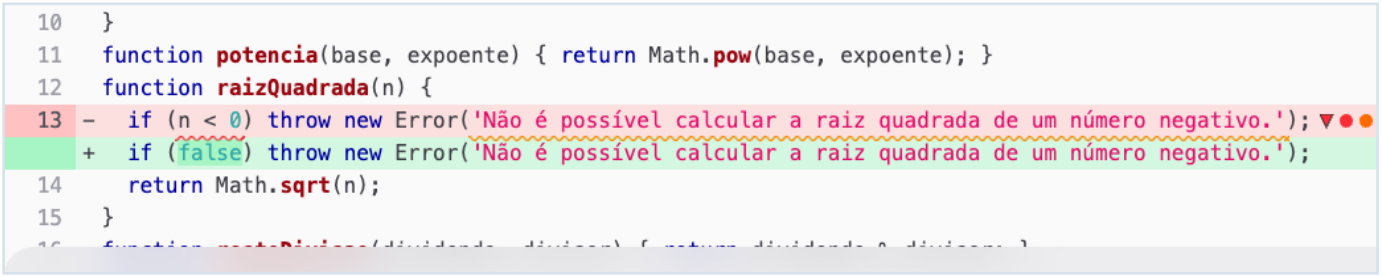
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	73.71	78.11	154	44	3	12	0	0	0	157	56	213
operacoes.js	73.71	78.11	154	44	3	12	0	0	0	157	56	213

A pontuação inicial foi calculada sobre todos os mutantes elegíveis. A cobertura de statements também era 85,41%; a de funções, 100%. Assim, até mesmo uma função chamada pelos testes podia esconder caminhos e retornos incorretos.

Repositório: github.com/JoaoRajao/operacoes-mutante

2. Três falhas reveladas pelo relatório

A. Raiz quadrada - guarda removida



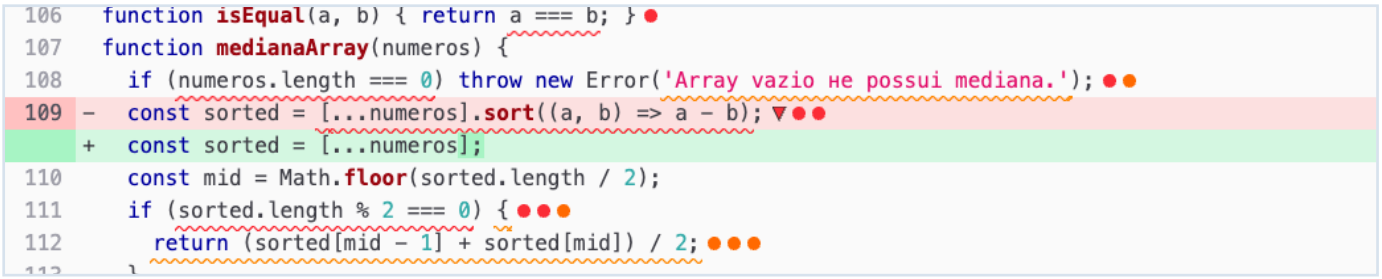
Mutação: if (n < 0) virou if (false). O teste original só usava 16 e não exercitava a exceção. Novo caso: raizQuadrada(-4) exige a mensagem de erro; 0 confirma a fronteira aceita. O mutante passou de Survived para Killed.

B. Primalidade - rejeição de composto removida



Mutação: if (n % i === 0) virou if (false). O teste original só verificava 7 (primo). Novos casos 4 e 9 exigem false; 0, 1 e 2 cobrem limites. O mutante agora é morto por um número composto.

C. Mediana - ordenação eliminada



Mutação: [...numeros].sort(...) virou apenas [...numeros]. A entrada original já estava ordenada. Novos casos [9, 1, 5] e [8, 1, 4, 2] tornam a ordenação observável e também exercitam os ramos ímpar e par.

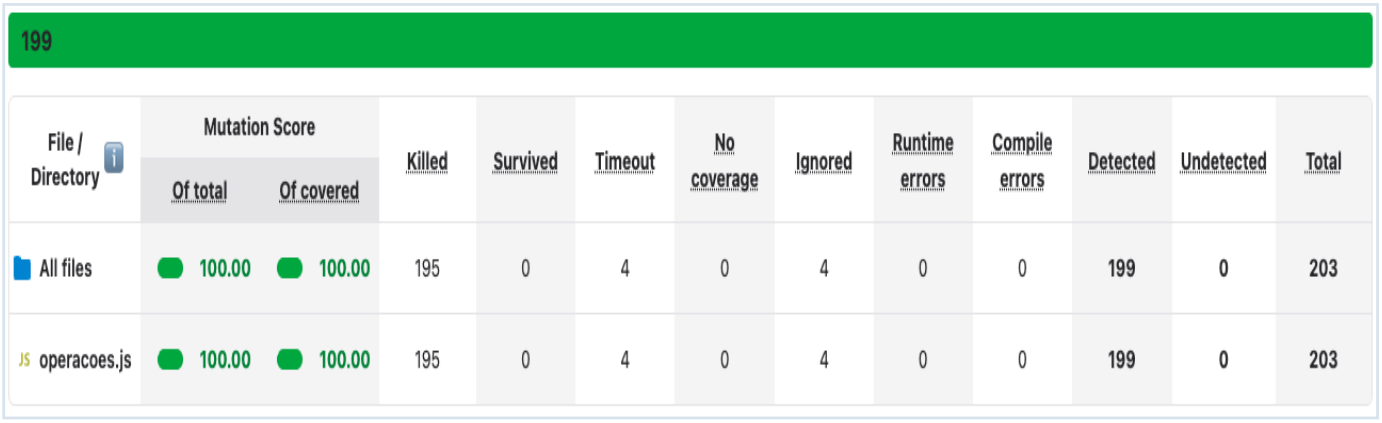
Solução, resultados e conclusão

3. Suíte aprimorada

Foram acrescentados 14 testes direcionados (50 para 64). Eles verificam erros e mensagens, arrays vazios, limites de paridade e primalidade, clamp, comparações falsas/iguais, temperaturas não nulas e medianas desordenadas. Todos os 64 testes passam; a cobertura final de linhas e ramos é 100%.

4. Pontuação final e tratamento dos equivalentes

Somente com os novos testes, sem alterar o código-fonte, o score subiu de 73,71% para 96,71%: 203 mortos, 3 timeouts e 7 sobreviventes.
Os 7 sobreviventes eram equivalentes para entradas válidas: ramos redundantes de fatorial e produtoArray e comparações < / <=, > / >= nas bordas do clamp.
Removemos os dois ramos redundantes sem mudar o resultado; 4 mutantes de EqualityOperator no clamp foram marcados Ignored com justificativa no código.



199

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	100.00	100.00	195	0	4	0	4	0	0	199	0	203
operacoes.js	100.00	100.00	195	0	4	0	4	0	0	199	0	203

Score final do Stryker: 100,00% dos 199 mutantes considerados (195 mortos, 4 timeouts). O relatório mostra ainda 4 Ignored e nenhum sobrevivente.
Logo, o 100% é o score ajustado, não o score da suíte isolada. A melhora atribuível apenas aos testes é de 23,00 pontos percentuais.

5. Conclusão

O teste de mutação detectou lacunas que a cobertura de linhas não mostrou.
Casos de fronteira e asserções específicas transformaram execução em verificação.
Mutantes equivalentes exigem análise humana e tratamento explícito no relatório.

Evidências: evidencias/inicial, evidencias/sem-exclusoes e evidencias/final.

Fonte: StrykerJS - stryker-mutator.io/docs/stryker-js/disable-mutants/

Código e relatório: github.com/JoaoRajao/operacoes-mutante